

Section 99 Editorial — Master Frontier DP Mixed Contest

A. Checksum Digit Sum

How to find it

Take the product of tight digit DP with a digit-sum counter and a remainder automaton.

Correctness

Decimal append updates the remainder exactly, so accepted paths are precisely the numbers satisfying both conditions.

Complexity

$O(19 * S * M * 10)$ per bound.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
string digits;
int target,mod_value;
map<tuple<int,int,int,bool>,ll>memo;
ll dfs(int pos,int sum,int rem,bool tight) {
    if(sum>target)return 0;
    if(pos==(int)digits.size())return sum==target&&rem==0;
    auto key=tuple {
        pos,sum,rem,tight
    };
    ;
    if(!tight&&memo.count(key))return memo[key];
    int limit=tight?digits[pos]-'0':9;
    ll answer=0;
    for(int d=0;d<=limit;d++)answer+=dfs(pos+1,sum+d,(rem*10+d)%mod_value,tight&&d==limit);
    if(!tight)memo[key]=answer;
    return answer;
}
ll count_to(ll n,int s,int m) {
    if(n<0)return 0;
    digits=to_string(n);
    target=s;
    mod_value=m;
    memo.clear();
    return dfs(0,0,0,true);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
```

```

    ll l,r;
    int s,m;
    cin>>l>>r>>s>>m;
    cout<<count_to(r,s,m)-count_to(l-1,s,m)<<'\n';
}
}

```

Python

```

import sys
from functools import lru_cache

def count(n, target, mod):
    if n < 0:
        return 0
    digits = list(map(int, str(n)))

    @lru_cache(None)
    def dp(pos, total, rem, tight):
        if total > target:
            return 0
        if pos == len(digits):
            return int(total == target and rem == 0)
        limit = digits[pos] if tight else 9
        return sum((dp(pos + 1, total + d, (rem * 10 + d) % mod, tight and d == limit) for d in range(limit + 1)))
    return dp(0, 0, 0, True)

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(1, len(d), 4):
        out.append(str(count(d[i + 1], d[i + 2], d[i + 3]) - count(d[i] - 1, d[i + 2], d[i + 3])))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

B. Serial Filter

How to find it

Combine the KMP automaton with a bounded digit-sum dimension.

Correctness

KMP remembers exactly the suffix relevant to a future forbidden match, while the sum dimension is exact.

Complexity

$O(19 * |P| * S * 10)$ per query.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll=long long;
string digits,pattern;
vector<int>failure;

```

```

int target;
map<tuple<int,int,int,bool,bool>,ll>memo;
int step(int j,int d) {
    char c='0'+d;
    while(j&&pattern[j]!=c)j=failure[j-1];
    if(pattern[j]==c)j++;
    return j;
}
ll dfs(int pos,int j,int sum,bool tight,bool started) {
    if(sum>target)return 0;
    if(pos==(int)digits.size())return sum==target&&(started||pattern!="0");
    auto key=tuple {
        pos,j,sum,tight,started
    };
    ;
    if(!tight&&memo.count(key))return memo[key];
    int limit=tight?digits[pos]-'0':9;
    ll answer=0;
    for(int d=0;d<=limit;d++) {
        bool next_tight=tight&&d==limit;
        if(!started&&d==0)answer+=dfs(pos+1,0,sum,next_tight,false);
        else {
            int next=step(j,d);
            if(next<(int)pattern.size())answer+=dfs(pos+1,next,sum+d,next_tight,true);
        }
    }
    if(!tight)memo[key]=answer;
    return answer;
}
ll solve(ll n,string p,int s) {
    digits=to_string(n);
    pattern=p;
    target=s;
    failure.assign(p.size(),0);
    for(int i=1;i<(int)p.size();i++) {
        int j=failure[i-1];
        while(j&&p[i]!=p[j])j=failure[j-1];
        if(p[i]==p[j])j++;
        failure[i]=j;
    }
    memo.clear();
    return dfs(0,0,0,true,false);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
        ll n;
        string p;
        int s;
        cin>>n>>p>>s;
        cout<<solve(n,p,s)<<'\\n';
    }
}

```

Python

```

import sys
from functools import lru_cache

```

```

def count(n, pattern, target):
    digits = list(map(int, str(n)))
    p = list(map(int, pattern))
    fail = [0] * len(p)
    for i in range(1, len(p)):
        j = fail[i - 1]
        while j and p[i] != p[j]:
            j = fail[j - 1]
        if p[i] == p[j]:
            j += 1
        fail[i] = j

    def step(j, d):
        while j and p[j] != d:
            j = fail[j - 1]
        if p[j] == d:
            j += 1
        return j

    @lru_cache(None)
    def dp(pos, j, total, tight, started):
        if total > target:
            return 0
        if pos == len(digits):
            return int(total == target and (started or pattern != '0'))
        limit = digits[pos] if tight else 9
        answer = 0
        for d in range(limit + 1):
            nt = tight and d == limit
            if not started and d == 0:
                answer += dp(pos + 1, 0, total, nt, False)
            else:
                nj = step(j, d)
                if nj < len(p):
                    answer += dp(pos + 1, nj, total + d, nt, True)
        return answer
    return dp(0, 0, 0, True, False)

def main():
    data = sys.stdin.buffer.read().split()
    out = []
    for i in range(1, len(data), 3):
        out.append(str(count(int(data[i]), data[i + 1].decode(), int(data[i + 2]))))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

C. Repeating Domino Tower

How to find it

Compose one matrix per obstacle row into a period matrix, then exponentiate it.

Correctness

Row matrices enforce local obstacles; composition represents one period and exponentiation represents exactly H periods.

Complexity

$O(P * 2^{(3W)} + 2^{(3W)} \log H)$ with dense matrices.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
const int MOD=1e9+7;
using Matrix=vector<vector<int>>>;
Matrix multiply(const Matrix&a,const Matrix&b) {
    int n=a.size();
    Matrix c(n,vector<int>(n));
    for(int i=0;i<n;i++)for(int k=0;k<n;k++)if(a[i][k])for(int j=0;j<n;j++)c[i][j]=(c[i][j]+(ll)a[i][k]*b[k][j])%MOD;
    return c;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long h;
    int p,w;
    cin>>h>>p>>w;
    vector<string>rows(p);
    for(auto&row:rows)cin>>row;
    int size=1<<w;
    Matrix block(size,vector<int>(size));
    for(int i=0;i<size;i++)block[i][i]=1;
    for(auto row:rows) {
        int blocked=0;
        for(int c=0;c<w;c++)if(row[c]=='#')blocked|=1<<c;
        Matrix transition(size,vector<int>(size));
        for(int incoming=0;incoming<size;incoming++)if(!(incoming&blocked)) {
            function<void(int,int,int)>fill=[&](int c,int used,int outgoing) {
                if(c==w) {
                    transition[incoming][outgoing]++;
                    return;
                }
                int bit=1<<c;
                if((used|blocked)&bit)fill(c+1,used,outgoing);
                else {
                    if(c+1<w&&!((used|blocked)&(bit<<1)))fill(c+2,used|bit|(bit<<1),outgoing);
                    fill(c+1,used|bit,outgoing|bit);
                }
            };
            fill(0,incoming,0);
        }
        block=multiply(block,transition);
    }
    Matrix answer(size,vector<int>(size));
    for(int i=0;i<size;i++)answer[i][i]=1;
    while(h) {
        if(h&1)answer=multiply(answer,block);
        block=multiply(block,block);
        h>>=1;
    }
    cout<<answer[0][0]<<'\\n';
}
```

Python

```
import sys
MOD = 1000000007
```

```

def mul(a, b):
    n = len(a)
    c = [[0] * n for _ in range(n)]
    for i in range(n):
        for k, x in enumerate(a[i]):
            if x:
                for j, y in enumerate(b[k]):
                    c[i][j] = (c[i][j] + x * y) % MOD
    return c

def main():
    data = sys.stdin.buffer.read().split()
    h, p, w = map(int, data[:3])
    rows = [x.decode() for x in data[3:]]
    size = 1 << w
    block = [[int(i == j) for j in range(size)] for i in range(size)]
    for row in rows:
        blocked = sum((row[c] == '#') << c for c in range(w))
        mat = [[0] * size for _ in range(size)]
        for incoming in range(size):
            if incoming & blocked:
                continue

            def fill(c, used, out):
                if c == w:
                    mat[incoming][out] += 1
                    return
                bit = 1 << c
                if (used | blocked) & bit:
                    fill(c + 1, used, out)
                else:
                    if c + 1 < w and (not (used | blocked) & bit << 1):
                        fill(c + 2, used | bit | bit << 1, out)
                        fill(c + 1, used | bit, out | bit)
                    fill(0, incoming, 0)
            block = mul(block, mat)
        result = [[int(i == j) for j in range(size)] for i in range(size)]
        while h:
            if h & 1:
                result = mul(result, block)
                block = mul(block, block)
            h //= 2
        print(result[0][0])
if __name__ == '__main__':
    main()

```

D. Connected Reserve

How to find it

Use connectivity labels, but prohibit skipping every T cell.

Correctness

Every required cell is selected, and the disappearance rule accepts exactly selections ending in one component.

Complexity

Exponential in W and polynomial in H.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
const int MOD=1e9+7;
using State=pair<vector<int>,bool>;
vector<int>canon(vector<int>a) {
    map<int,int>mp;
    int nxt=1;
    for(int&x:a)if(x) {
        if(!mp.count(x))mp[x]=nxt++;
        x=mp[x];
    }
    return a;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h,w;
    cin>>h>>w;
    vector<string>g(h);
    for(auto&s:g)cin>>s;
    map<State,int>dp;
    dp[ {
        vector<int>(w),false
    }
    ]=1;
    for(int r=0;r<h;r++)for(int c=0;c<w;c++) {
        map<State,int>nd;
        for(auto[state,value]:dp) {
            auto labels=state.first;
            bool closed=state.second;
            int up=labels[c],left=c?labels[c-1]:0;
            auto z=labels;
            z[c]=0;
            bool vanished=up&&find(z.begin(),z.end(),up)==z.end();
            if(g[r][c]!='T'&&(!vanished||(!closed&&count_if(z.begin(),z.end(),[](int x){return x;})==0))) {
                State key= {
                    canon(z),closed||vanished
                };
                ;
                nd[key]=(nd[key]+value)%MOD;
            }
            if(g[r][c]!='#'&&!closed) {
                z=labels;
                if(left&&up&&left!=up)for(int&x:z)if(x==up)x=left;
                z[c]=left?left:(up?up:*max_element(z.begin(),z.end())+1);
                State key= {
                    canon(z),false
                };
                ;
                nd[key]=(nd[key]+value)%MOD;
            }
        }
        dp.swap(nd);
    }
    ll answer=0;
    for(auto[state,value]:dp) {
        set<int>s(state.first.begin(),state.first.end());
        s.erase(0);
        if(state.second||s.size()==1)answer+=value;
    }
}
```

```

    cout<<answer%MOD<<'\n';
}

```

Python

```

import sys
MOD = 1000000007

def canon(a):
    mp = {}
    nxt = 1
    out = []
    for x in a:
        if x and x not in mp:
            mp[x] = nxt
            nxt += 1
        out.append(mp.get(x, 0))
    return tuple(out)

def main():
    data = sys.stdin.buffer.read().split()
    h, w = map(int, data[:2])
    g = [x.decode() for x in data[2:]]
    dp = {(tuple([0] * w), False): 1}
    for r in range(h):
        for c in range(w):
            nd = {}
            for (labels, closed), value in dp.items():
                old = list(labels)
                up = old[c]
                left = old[c - 1] if c else 0
                new = old[:]
                new[c] = 0
                vanished = up and up not in new
                if g[r][c] != 'T' and (not vanished or (not any(new) and (not closed))):
                    key = (canon(new), closed or bool(vanished))
                    nd[key] = (nd.get(key, 0) + value) % MOD
                if g[r][c] != '#' and (not closed):
                    new = old[:]
                    if left and up and (left != up):
                        new = [left if x == up else x for x in new]
                    new[c] = left or up or max(new, default=0) + 1
                    key = (canon(new), False)
                    nd[key] = (nd.get(key, 0) + value) % MOD
            dp = nd
    answer = 0
    for (labels, closed), value in dp.items():
        if closed or len(set(labels) - {0}) == 1:
            answer = (answer + value) % MOD
    print(answer)
if __name__ == '__main__':
    main()

```

E. Terminal Backbone

How to find it

Compute every terminal-mask Steiner row once and answer queries from those rows.

Correctness

The subset-merge and shortest-path proof applies independently to every queried mask.

Complexity

$O(3^k n + 2^k m \log n + qn)$.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
const ll INF=4e18;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m,k,queries;
    cin>>n>>m>>k>>queries;
    vector<vector<pair<int,int>>>g(n);
    while(m--) {
        int u,v,w;
        cin>>u>>v>>w;
        --u;
        --v;
        g[u].push_back({v,w});
        g[v].push_back({u,w});
    }
    vector<vector<ll>>dp(1<<k,vector<ll>(n,INF));
    for(int i=0,v;i<k;i++)cin>>v,dp[1<<i][v-1]=0;
    for(int mask=1;mask<1<<k;mask++) {
        for(int sub=(mask-1)&mask;sub;sub=(sub-1)&mask) {
            int other=mask^sub;
            if(sub<other)for(int v=0;v<n;v++)dp[mask][v]=min(dp[mask][v],dp[sub][v]+dp[other][v]);
        }
        priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>>q;
        for(int v=0;v<n;v++)if(dp[mask][v]<INF)q.push({dp[mask][v],v});
        while(!q.empty()) {
            auto[d,u]=q.top();
            q.pop();
            if(d!=dp[mask][u])continue;
            for(auto[v,w]:g[u])if(d+w<dp[mask][v])dp[mask][v]=d+w,q.push({d+w,v});
        }
    }
    while(queries--) {
        int mask;
        cin>>mask;
        ll ans=*min_element(dp[mask].begin(),dp[mask].end());
        if(ans==INF)cout<<"IMPOSSIBLE\n";
        else cout<<ans<<'\\n';
    }
}
```

Python

```
import sys, heapq

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, k, queries = d[:4]
```

```

at = 4
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, w = d[at:at + 3]
    at += 3
    u -= 1
    v -= 1
    g[u].append((v, w))
    g[v].append((u, w))
terminals = [x - 1 for x in d[at:at + k]]
inf = 10 ** 30
dp = [[inf] * n for _ in range(1 << k)]
for i, v in enumerate(terminals):
    dp[1 << i][v] = 0
for mask in range(1, 1 << k):
    sub = mask - 1 & mask
    while sub:
        other = mask ^ sub
        if sub < other:
            for v in range(n):
                dp[mask][v] = min(dp[mask][v], dp[sub][v] + dp[other][v])
        sub = sub - 1 & mask
    q = [(x, v) for v, x in enumerate(dp[mask]) if x < inf]
    heapq.heapify(q)
    while q:
        cost, u = heapq.heappop(q)
        if cost != dp[mask][u]:
            continue
        for v, w in g[u]:
            if cost + w < dp[mask][v]:
                dp[mask][v] = cost + w
                heapq.heappush(q, (cost + w, v))
    out = []
    for mask in d[at + k:at + k + queries]:
        ans = min(dp[mask])
        out.append('IMPOSSIBLE' if ans == inf else str(ans))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

F. Decomposition Profit

How to find it

A vertex cover is the complement of an independent set.

Correctness

The complement covers every edge exactly when the chosen set contains no edge, so the answer is total weight minus maximum independent-set weight.

Complexity

$O(t * 2^w * w)$ expected map work.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll=long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m;
    cin>>n>>m;
    vector<ll>w(n);
    ll total=0;
    for(ll&x:w) {
        cin>>x;
        total+=x;
    }
    vector<unsigned long long>adj(n);
    while(m--) {
        int u,v;
        cin>>u>>v;
        --u;
        --v;
        adj[u]|=1ULL<<v;
        adj[v]|=1ULL<<u;
    }
    int t;
    cin>>t;
    vector<unsigned long long>bag(t);
    vector<map<unsigned long long,ll>>dp(t);
    for(int i=0;i<t;i++) {
        char type;
        cin>>type;
        if(type=='L')dp[i][0]=0;
        else if(type=='I') {
            int child,v;
            cin>>child>>v;
            --child;
            --v;
            bag[i]=bag[child]|1ULL<<v;
            dp[i]=dp[child];
            for(auto[mask,val]:dp[child])if(!(mask&adj[v])) {
                auto nextmask=mask|1ULL<<v;
                ll candidate=val+w[v];
                if(!dp[i].count(nextmask)||candidate>dp[i][nextmask])dp[i][nextmask]=candidate;
            }
        }
        else if(type=='F') {
            int child,v;
            cin>>child>>v;
            --child;
            --v;
            bag[i]=bag[child]&~(1ULL<<v);
            for(auto[mask,val]:dp[child]) {
                auto nextmask=mask&~(1ULL<<v);
                if(!dp[i].count(nextmask)||val>dp[i][nextmask])dp[i][nextmask]=val;
            }
        }
        else {
            int a,b;
            cin>>a>>b;
            --a;
            --b;
            bag[i]=bag[a];
            for(auto[mask,val]:dp[a])if(dp[b].count(mask)) {
                ll duplicate=0;

```

```

        for(int v=0;v<n;v++)if(mask>>v&1)duplicate+=w[v];
        dp[i][mask]=val+dp[b][mask]-duplicate;
    }
}
}
cout<<total-dp.back()[0]<<'\n';
}

```

Python

```

import sys

def main():
    lines = sys.stdin.buffer.readlines()
    n, m = map(int, lines[0].split())
    weight = list(map(int, lines[1].split()))
    adj = [0] * n
    for line in lines[2:2 + m]:
        u, v = map(int, line.split())
        u -= 1
        v -= 1
        adj[u] |= 1 << v
        adj[v] |= 1 << u
    t = int(lines[2 + m])
    dp = []
    bags = []
    for line in lines[3 + m:3 + m + t]:
        z = line.split()
        kind = z[0]
        if kind == b'L':
            bags.append(0)
            dp.append({0: 0})
        elif kind == b'I':
            child = int(z[1]) - 1
            v = int(z[2]) - 1
            bags.append(bags[child] | 1 << v)
            cur = dict(dp[child])
            for mask, val in dp[child].items():
                if not mask & adj[v]:
                    cur[mask | 1 << v] = max(cur.get(mask | 1 << v, -10 ** 30), val + weight[v])
            dp.append(cur)
        elif kind == b'F':
            child = int(z[1]) - 1
            v = int(z[2]) - 1
            bags.append(bags[child] & ~(1 << v))
            cur = {}
            for mask, val in dp[child].items():
                key = mask & ~(1 << v)
                cur[key] = max(cur.get(key, -10 ** 30), val)
            dp.append(cur)
        else:
            a = int(z[1]) - 1
            b = int(z[2]) - 1
            bags.append(bags[a])
            cur = {}
            for mask, val in dp[a].items():
                if mask in dp[b]:
                    cur[mask] = val + dp[b][mask] - sum((weight[v] for v in range(n) if mask >> v & 1))
            dp.append(cur)
    print(sum(weight) - dp[-1].get(0, max(dp[-1].values())))
if __name__ == '__main__':
    main()

```

Final frontier-DP audit

For every state, write the equivalence claim: any two processed histories with that state admit exactly the same future completions, with the same remaining cost or count. Then justify every forgetting event. Occupancy bits suffice for local tilings; automaton states summarize digit suffixes; canonical labels preserve connectivity; bag assignments preserve interaction through a tree decomposition.

Use literal exhaustive oracles on the smallest domains: enumerate bounded integers, domino placements, cell subsets, connecting edge subsets, and vertex subsets. Include empty masks, all obstacles, width one, a component losing its last frontier cell, terminal subsets of size zero and one, and transfer-matrix exponents zero and one. These boundary cases test the compression proof more directly than large random instances.